

Limited-Angle CT Reconstruction

Computational Imaging – Group D Project

Alessio Manieri
Daniele Silvestri
University of Bologna

September 2, 2026

Abstract

This project addresses the problem of limited-angle CT reconstruction on the Mayo dataset, comparing methods belonging to different families: a variational method (Total Variation), an end-to-end method based on a neural network (UNet), a hybrid method (Weighted TV), and a fourth generative method (GAN with Diffusion Posterior Sampling). All four were evaluated on the same test set (327 slices from a single patient, 4 angular configurations). The hybrid Weighted TV method obtains the best metrics (PSNR and SSIM with respect to the ground truth) in all 4 configurations, with a modest but systematic gain over plain TV; the UNet, trained to imitate the TV reconstruction rather than the ground truth, closely matches its own target but does not exceed the quality of TV on the final metric, for structural reasons discussed in Section 8. The generative method, despite two generator training attempts and a dedicated tuning of the sampling loop, remains clearly below the other three: an informative result on the practical limits of a generator trained on a dataset of limited size, discussed in Section 6.9.

1 Introduction

Computed tomography (CT) reconstructs an image x from a set of projections (sinogram) y , acquired along different angles around the object. The problem is formulated as a linear inverse problem:

$$y_\delta = Kx + e, \quad (1)$$

where K is the projection operator (discrete Radon transform, parallel-beam geometry), and e represents the acquisition noise, here modeled as additive Gaussian noise proportional to the norm of the sinogram.

In the *limited-angle* case, the projections do not cover the entire angular range $[0^\circ, 180^\circ]$ but only a restricted subset of angles. This makes the problem strongly ill posed: the operator K has a spectrum that decays rapidly in the directions corresponding to the missing angles (*missing wedge*), and direct inversion (e.g. Filtered Back Projection, FBP) produces characteristic streak/butterfly artifacts, in addition to amplifying the noise. It is therefore necessary to introduce prior information on the solution, either through explicit regularization (e.g. Total Variation) or through priors learned from data (neural networks, generative models).

1.1 Task definition

The project requires comparing several families of methods on the same task, with the same degraded data:

- **Dataset:** Mayo (low-dose CT), images resized to 256×256 ;
- **Geometry:** parallel-beam projection, with 4 angular configurations:

- 90 equally spaced angles in $[-45^\circ, 45^\circ]$;
- 45 equally spaced angles in $[-45^\circ, 45^\circ]$;
- 30 equally spaced angles in the open interval $(-30^\circ, 30^\circ)$;
- 15 equally spaced angles in the open interval $(-30^\circ, 30^\circ)$;
- **Noise:** additive Gaussian noise on the sinogram, with relative level 0.005;
- **Methods compared:** a variational method (TV), an end-to-end method (UNet), a hybrid method (Weighted TV), and a generative method (GAN with data-guided sampling, GAN+DPS), the latter the most demanding to get right, and, as we will see, also the weakest of the four on the final metric (Section 6);
- **Metrics:** PSNR and SSIM, computed with respect to the ground truth.

1.2 Document structure

Section 2 describes the dataset and the preprocessing pipeline. Section 3 formalizes the inverse problem and the generation of the degraded data. Section 4 describes the variational TV method. Section 5 describes the end-to-end method (UNet). Section 6 is dedicated to the generative method. Section 7 describes the hybrid Weighted TV method. Section 8 reports the final experimental comparison across all implemented methods. Section 9 closes with the conclusions.

2 Dataset and preprocessing

The dataset used is Mayo [2] (low-dose CT), organized by patient. The original images are provided in PNG format, 4-channel RGBA; a preliminary check confirmed that the three RGB channels are identical and the alpha channel is constant ($= 255$), so conversion to grayscale (a single channel) does not entail any loss of information.

Preprocessing pipeline (notebook `01_preprocessing.ipynb`):

1. reading of the original images and selection of a single channel (grayscale);
2. resize to 256×256 (`skimage.transform.resize`, with anti-aliasing, original range and dtype preserved);
3. train/test split already present in the structure of the original dataset (3306 images in train, 327 in test);
4. creation of a validation set by moving 2 entire patients out of the train set (fixed seed $= 123$, patients C002 and C027; of the 10 patients in the original train set, 8 remain for training and 2 for validation), to avoid data leakage between slices of the same patient.

Composition of the test set. It is important to clarify a structural feature of the dataset that affects the interpretation of *all* the quantitative results in Section 8: the test split provided by the Mayo dataset for this project corresponds to a **single patient** (identifier C081), whose 327 slices exactly match the total of 327 test images reported above. All evaluation pipelines (TV, UNet, Weighted TV) are therefore evaluated on the same identical 327 slices, guaranteeing a homogeneous comparison across methods; however, the final metrics describe reconstruction quality on a single CT volume, not on a sample representative of multiple patients. The conclusions on the relative superiority of the methods (Section 8) should therefore be read with this caveat in mind: a second test patient, if available, would be the most direct way to verify that the observed ranking between methods is not an artifact of the single case.

3 Problem formulation and generation of the degraded data

3.1 Projection operator

The operator K in (1) is implemented via `IPPy.operators.CTProjector` (library `IPPy` [1], based on the `ASTRA Toolbox` [?] for computing the Radon transform and its adjoint), with "parallel" geometry. For each of the 4 angular configurations, a distinct projector is instantiated, shared across all images of the same configuration (initializing the `ASTRA` operator has a non-negligible cost).

3.2 Sinogram generation

For each image in the resized dataset (train/validation/test) and for each of the 4 angular configurations (notebook `02_synogram_generation.ipynb`):

1. computation of the clean sinogram Kx ;
2. addition of Gaussian noise (`IPPy.utilities.gaussian_noise`, level 0.005, fixed seed = 42 for reproducibility, set once for the entire generation and not per individual image);
3. saving of the noisy sinogram y_δ in `.npy` format, in a folder structure `sinograms/<split>/<n_angles>/<pat`

Figure 1 shows an example of a generated sinogram (90-angle configuration).

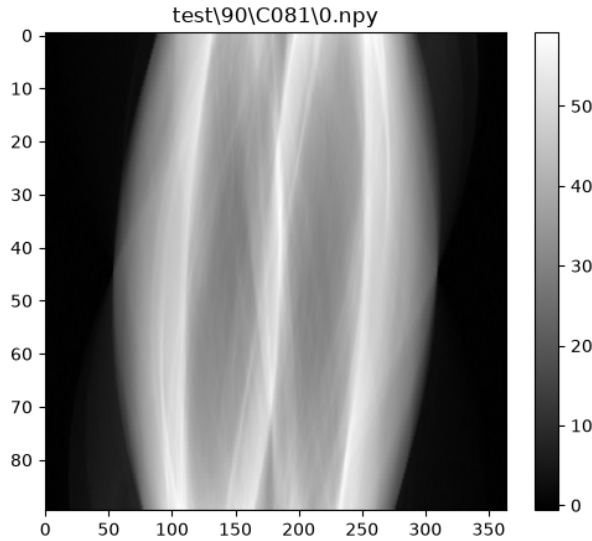


Figure 1: Example of a noisy sinogram generated for the 90-angle configuration (vertical axis: projection angle, horizontal axis: position on the detector).

3.3 Baseline: Filtered Back Projection

As a naive reference (as well as a consistency check between sinogram and operator), the direct FBP reconstruction of every sinogram was also computed (notebook `04_preprocessing.ipynb`), a cheap operation and therefore run on CPU for the entire dataset.

The comparison between the value ranges of FBP and of the TV reconstructions (Section 4) on a sample of 300 images quantified the extent of the excursions outside the expected range $[0, 1]$: FBP shows negative pixels in **all** 300 sampled images (100%), with an average fraction of negative pixels per image of 17.5% (worst excursion observed: -1.21) and pixels above 1 in 78.7% of the images (236/300, worst excursion $+0.95$). The TV reconstruction, by construction of the solver (constraint $x \geq 0$ explicitly imposed), never shows negative pixels (0/300), but is

not exempt from the opposite problem: TV too exceeds the value 1 in 73.7% of the images (221/300), although with excursions that are typically smaller (worst +0.78 versus +0.95 for FBP). The impression that TV regularization alone solves the range problem should therefore be corrected: it solves it only from the lower side (by construction of the solver), not from the upper one, which is why in all evaluation pipelines (Section 8) an explicit clipping to $[0, 1]$ is applied downstream of every reconstruction, TV included. Figure 2 shows an example of the butterfly/streak artifacts typical of FBP in this regime, compared with the ground truth.

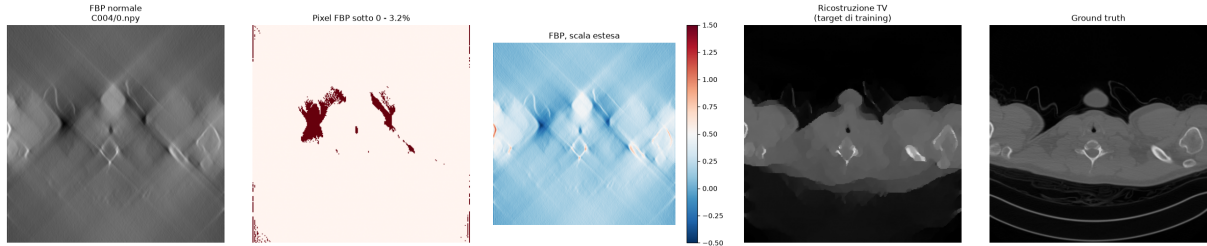


Figure 2: FBP reconstruction (90 angles) compared with the ground truth: map of pixels with negative value, extended-scale visualization to highlight the *missing wedge* artifacts, TV reconstruction (training target of the UNet), and original image.

4 Variational method: Total Variation

4.1 Formulation

The TV reconstruction is obtained by solving the constrained problem (Morozov discrepancy principle):

$$x^* \in \arg \min_x \text{TV}(x) \quad \text{s.t.} \quad \|Kx - y_\delta\|_2 \leq \varepsilon, \quad (2)$$

with $\varepsilon = \text{NOISE_LEVEL} \cdot \|y_\delta\|_2$, consistent with the noise level used when generating the sinograms (Section 3).

4.2 Algorithm

Problem (2) is solved with the Chambolle-Pock primal-dual algorithm [?], via the solver `IPPy.solvers.ChambollePockTpVConstrained`, with Total Variation regularizer [?]. Main parameters:

- `lmbda`: TV regularization weight (tuned per configuration, Section 4.3);
- `epsilon`: radius of the discrepancy constraint, as above;
- `starting_point`: zero image;
- `p=1`: disables the solver’s reweighting (standard isotropic TV, not pixel-wise weighted; the case $p < 1$, used instead in Section 7, activates a reweighting that is not needed here);
- `maxiter`: maximum number of iterations (Section 4.3).

At every iteration the solver also returns the evolution of the objective function, of the relative error (RE) with respect to the ground truth, and of PSNR and SSIM (`IPPy.utilities.metrics`), useful both for debugging and for hyperparameter tuning.

The main steps of the algorithm (checked against the implementation in `IPPy.solvers.ChambollePockTpVConstrained`) are reported in Algorithm 1. In addition to the primal variable x (the reconstructed image), the

method keeps two dual variables: p , associated with the data-discrepancy constraint, and q , associated with the TV regularization term, computed via the discrete gradient operator D . At each iteration both dual variables are updated and projected onto their own constraint (respectively a ball of radius $\sigma\varepsilon$ and, pixel by pixel, a ball of radius λ), then the primal variable is updated with a descent step followed by the non-negativity projection $x \geq 0$; finally an extrapolation step (*over-relaxation*) speeds up convergence. The steps τ, σ are set once at the beginning, by estimating the spectrum of the combined operator $[K; D]$ with the power method.

Algorithm 1 Chambolle-Pock primal-dual algorithm for constrained TV reconstruction (Eq. (2))

Require: sinogram y_δ , projection operator K , discrete gradient operator D , ε , λ , **maxiter**

```

1: Estimate  $\tau, \sigma$  from the spectrum of  $[K; D]$  (power method);  $\theta \leftarrow 1$ 
2:  $x^0 \leftarrow \mathbf{0}$ ,  $\bar{x}^0 \leftarrow x^0$ ,  $p^0 \leftarrow \mathbf{0}$ ,  $q^0 \leftarrow \mathbf{0}$ 
3: for  $k = 0, 1, \dots, \text{maxiter} - 1$  do
4:    $p^{k+1} \leftarrow \text{proj}_{\|\cdot\|_2 \leq \sigma\varepsilon} (p^k + \sigma(K\bar{x}^k - y_\delta))$  ▷ dual of the data constraint
5:    $q^{k+1} \leftarrow \text{proj}_{\|\cdot\|_2 \leq \lambda} (q^k + \sigma D\bar{x}^k)$  ▷ dual of the TV term, pixel-wise
6:    $x^{k+1} \leftarrow \max(x^k - \tau(K^\top p^{k+1} + D^\top q^{k+1}), 0)$  ▷ primal update + non-negativity
7:    $\bar{x}^{k+1} \leftarrow x^{k+1} + \theta(x^{k+1} - x^k)$  ▷ extrapolation
8:   if data residual stabilized (Section 4.3) then
9:     stop
10:  end if
11: end for
```

Ensure: reconstruction x^{k+1}

4.3 Hyperparameter tuning

Tuning (notebook 03_TV_hyperparametertuning.ipynb) was carried out on a sample of 12 validation images for each angular configuration, separately for:

- **λ (regularization weight):** grid search over $\lambda \in \{10^{-3}, 3 \cdot 10^{-3}, 10^{-2}, 3 \cdot 10^{-2}, 5 \cdot 10^{-2}, 10^{-1}, 2 \cdot 10^{-1}, 5 \cdot 10^{-1}\}$, with **maxiter** = 150 (half of the value used in the final generation: sufficient for a *relative* comparison between candidates, not for absolute quality), choosing the value that maximizes a 50/50 weighted combination of mean PSNR and SSIM normalized to $[0, 1]$ (Figure 3);
- **number of iterations:** a single longer run (400 iterations) with the already-chosen λ (Figure 4).

On the chosen number of iterations. The convergence run shows that, in **all 4** angular configurations, both the mean PSNR and the mean SSIM are still slightly increasing at the end of the 400 tested iterations (the maximum, for every configuration, falls exactly at the last observed iteration): the curve therefore does not reach a clear plateau within the explored range, and the automatic selection script (which picks the iteration of the observed maximum) would consistently return **maxiter** = 400 for all configurations. The value actually used at final-generation time (Table 1) is however **maxiter** = 300, lower than the observed maximum: this is a **deliberate trade-off between quality and computational cost** over the entire dataset (train+validation+test, 4 angular configurations, run on a Colab GPU), not an actual stabilization of the curve. It is important to present this choice in the report for what it is (a computation-time constraint, not evidence of convergence), because describing it otherwise would be misleading with respect to the tuning data itself.

The chosen values are reported in Table 1.

Table 1: Hyperparameters chosen for the TV method, per angular configuration.

Configuration (n. angles)	λ_{TV}	maxiter
90	0.05	300
45	0.05	300
30	0.03	300
15	0.03	300

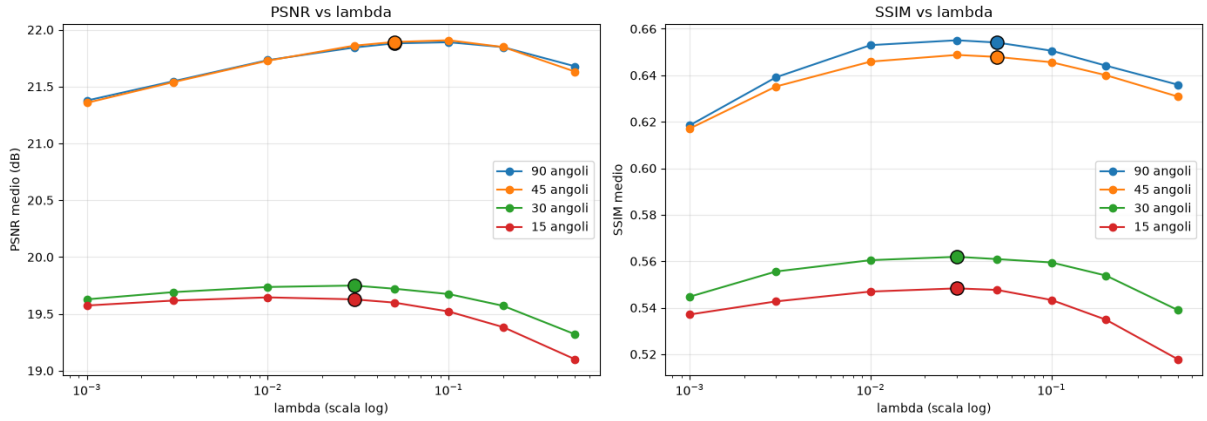


Figure 3: Mean PSNR and SSIM (validation) as λ varies, for each angular configuration. The circled points indicate the chosen λ . Source file: `tuning_results/lambda_tuning_curves.png` (rename/copy into `figures/` under this name, or update the path).

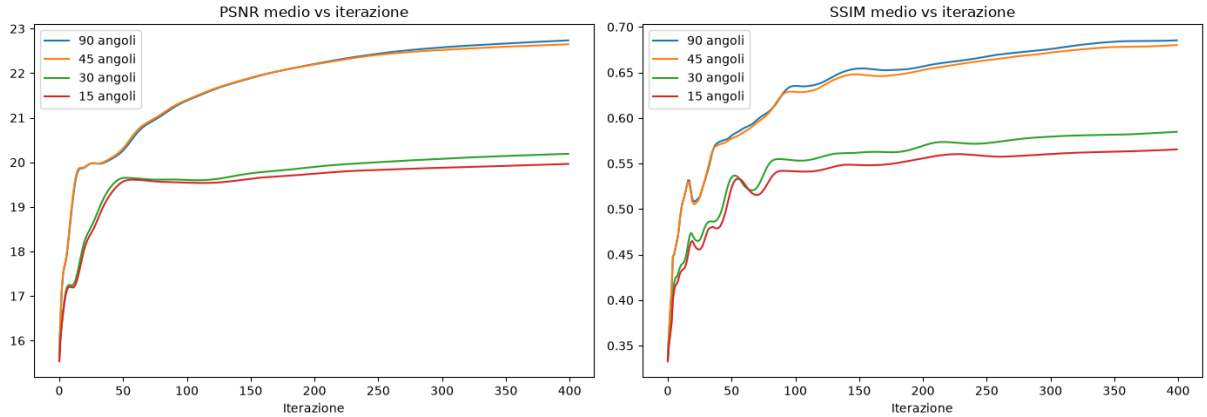


Figure 4: Convergence curves of mean PSNR and SSIM (validation) as the number of iterations varies, with optimal λ : note the absence of a clear plateau within the 400 tested iterations. Source file: `tuning_results/maxiter_convergence_curves.png`.

4.4 Implementation details

A first single-image test was run locally on CPU (`force_cpu=True`) to validate the correctness of the pipeline (notebook `03_init_test_TV.ipynb`, Figure 5). The TV reconstruction on the entire dataset (all splits, all angular configurations) was instead run on Google Colab with a GPU, for computation-time reasons:

- the IPPy library was installed directly from GitHub, with GPU support via `cupy-cuda12x` for data transfer in the ASTRA operator;
- a manual patch to IPPy’s `operators.py` file was required (the check for CuPy availability was not protected by a `try/except` block, causing a fallback to CPU even when CuPy was available), applied at runtime before importing `IPPy.operators`. The same patch was reused, unchanged, in all subsequent Colab notebooks (UNet training, Weighted TV generation and tuning, DPS generation and tuning);
- the sinograms were read from a local copy on the VM’s SSD (extracted from a zip uploaded to Google Drive), to avoid the I/O overhead of a mounted Drive;
- the produced reconstructions were incrementally synced to Google Drive (only files not yet present, every 100 images), so as not to lose the work done in case the Colab session was interrupted.



Figure 5: Example of a TV reconstruction (90-angle configuration): ground truth, input sinogram, reconstruction.

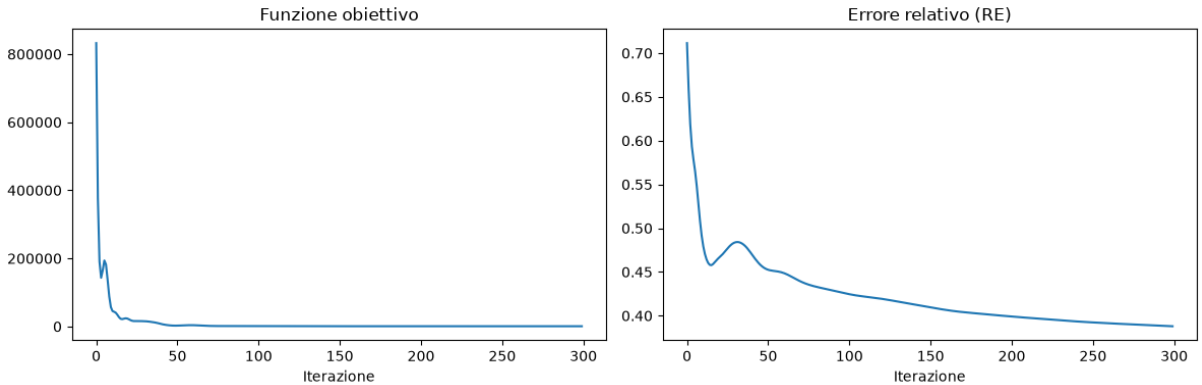


Figure 6: Evolution of the objective function and of the relative error (RE) during the iterations of the Chambolle-Pock solver, for the example in Figure 5.

5 End-to-end method

5.1 Architecture choice

Among the end-to-end architectures best suited to this type of task (UNet, ViT, NAF-Net) we chose a **UNet**. The choice is consistent with the nature of the task: the network must map a degraded image to a reconstructed image of the same spatial resolution (256×256), an image-to-image restoration problem in which it is crucial to preserve both the global anatomical structure and fine details (edges, small structures) damaged by *missing wedge* artifacts. The UNet’s encoder-decoder structure with skip connections is designed exactly for this: the residual connections allow the recovery of high-frequency details lost along the downsampling path, without having to relearn from scratch information already present in the input. Compared to ViT and NAF-Net, the UNet is also computationally lighter to train separately for each of the 4 angular configurations.

The architecture used (`IPPy.nn.models.UNet`) has the following configuration: intermediate channels $[32, 64, 128, 256]$ (3 down/up-sampling levels plus the bottleneck), 2 convolutional layers per block, residual blocks (`ResDownBlock/ResUpBlock`) in both encoder and decoder, final sigmoid activation: the latter explicitly constrains the network’s output to $[0, 1]$, consistent with the clipping adopted for FBP and TV (Section 3).

Figure 7 sketches the main path (encoder–bottleneck–decoder, which compresses the image into a compact representation and then expands it back) and the skip connections (which feed the decoder the high-resolution detail lost during compression, without having to pass through the bottleneck).

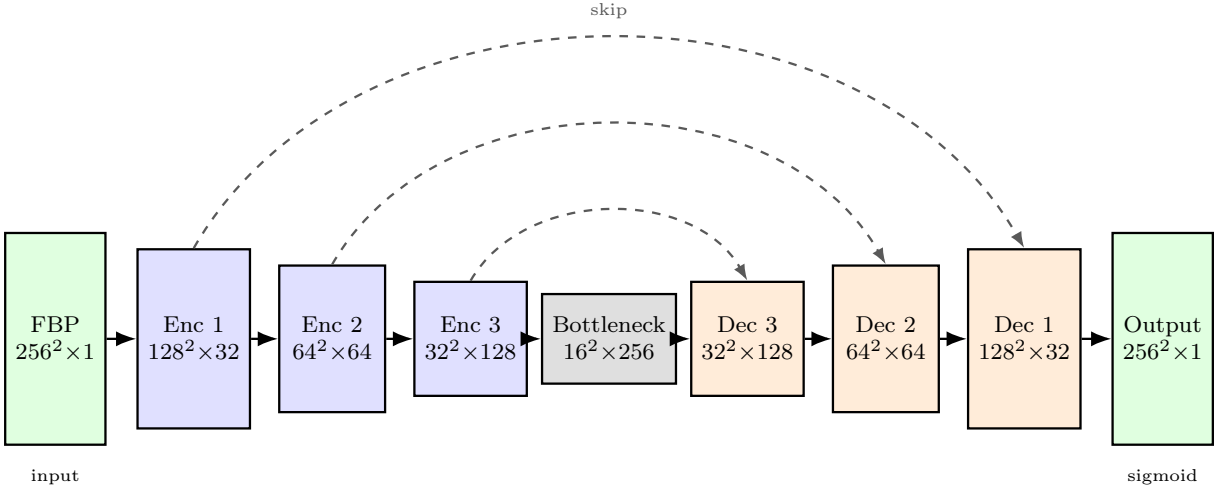


Figure 7: UNet architecture diagram: main path and skip connections.

5.2 Input, target, and formulation

The network learns a map $f_\theta : \mathbb{R}^{256 \times 256} \rightarrow \mathbb{R}^{256 \times 256}$ that refines a fast initial reconstruction into a higher-quality image:

- **Input:** FBP reconstruction (Section 3), computed directly and cheaply from each sinogram;
- **Target:** the reconstruction obtained with the variational TV method (Section 4), not the ground truth directly, a deliberate design choice, discussed further below. The network thus learns to approximate the effect of TV regularization with a single *forward* pass, instead of the 300 iterations of the Chambolle-Pock solver.

The ground truth (original, non-degraded image) is used only at final evaluation time (Section 8), to compare the network’s output both against the training objective (TV) and against the actual reconstruction quality.

This choice of target has a direct and non-obvious implication for the upper bound on the quality the network can reach with respect to the ground truth: since the UNet is trained to reproduce TV and not the ground truth, in the best case (perfect imitation) its quality with respect to the ground truth can match that of TV, but cannot exceed it by construction of the learning problem; any approximation error of the network with respect to its own target can therefore only worsen, not improve, the result relative to TV. This point is discussed quantitatively in Section 8, where it is also the natural bridge to the hybrid method of Section 7.

5.3 Training and model selection

Given the substantial difference in artifact patterns across the 4 angular configurations (number of angles and width of the covered range), a UNet was trained *separately for each configuration* (15, 30, 45, 90 angles), for a total of 4 independent trainings, each specialized on the corresponding degradation regime. Training run on Google Colab (GPU L4), notebook `04_training_unet.ipynb`.

Training hyperparameters (the same for all 4 configurations):

- **Loss:** weighted combination $1.0 \cdot \text{MSE} + 0.1 \cdot \text{SSIMLoss}$ (`IPPy.nn.losses.MixedLoss`): the SSIM term adds a component sensitive to local structure, with a reduced weight relative to the pure pixel-wise error;
- **Optimizer:** Adam, `learning_rate` = 10^{-4} ;
- **Learning rate:** chosen via a mini-tuning run on a subset of 300 training images, 5 epochs, comparing $\{10^{-3}, 10^{-4}, 10^{-5}\}$ (Figure 8). Final loss (MSE) after 5 epochs: $10^{-3} \rightarrow 0.0107$, $10^{-4} \rightarrow 0.0035$, $10^{-5} \rightarrow 0.0105$. The value 10^{-3} starts lower but quickly settles onto a plateau higher than 10^{-4} (the step is probably too large to refine the solution), while 10^{-5} is still decreasing and too slow to converge within the 5 trial epochs: 10^{-4} is the best trade-off between speed and final quality;
- **Epochs:** 25, **batch size:** 16;
- **Mixed precision** (`torch.amp`) to speed up training on GPU;
- **Model selection:** the validation loss is evaluated at every epoch; the weights (`model_weights.pth`) are overwritten only when the val loss improves on the minimum observed so far, so the final checkpoint always corresponds to the best model according to validation, not to the last epoch. The complete history (train/val loss, val PSNR, val SSIM per epoch) is saved at every epoch in `history.json`, which also allows training to be resumed in case the Colab session is interrupted.

5.4 Inference

The saved weights (one per configuration) were used to run inference on the FBP images of the test set (327 slices, Section 2), obtaining a UNet reconstruction for each slice and angular configuration (notebook `04_UNet_results.ipynb`). The resulting reconstructions are compared both against their own training target (TV) and against the ground truth; the numerical results and their interpretation (in particular why the UNet does not exceed TV despite an excellent fidelity to its own target) are discussed in Section 8.

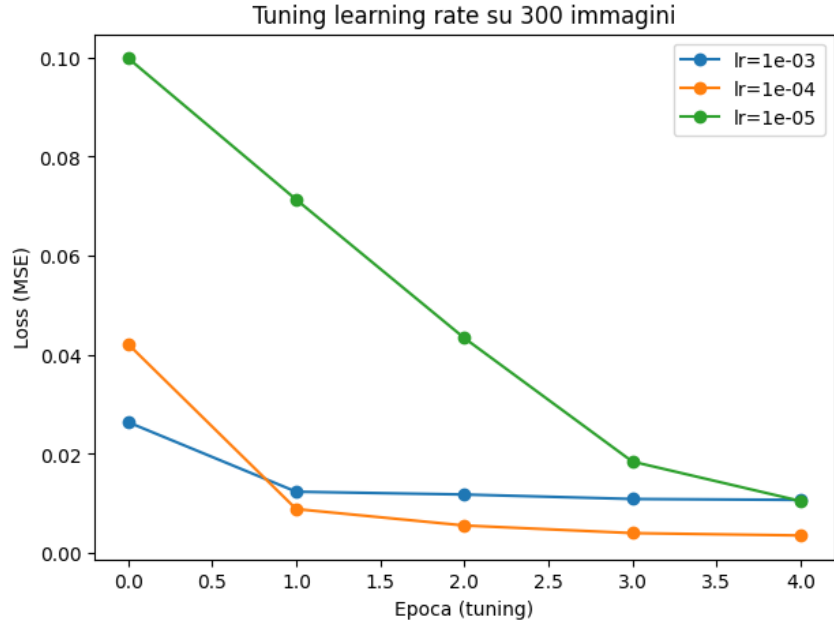


Figure 8: Learning rate tuning: loss (MSE) on a subset of 300 training images, 5 epochs, for the three candidates.

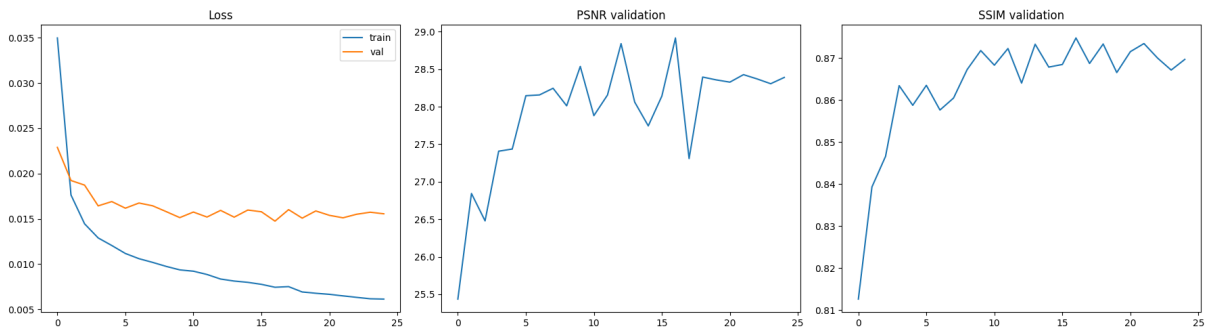


Figure 9: Train/val loss curves and val PSNR/SSIM during full training (25 epochs), 15-angle configuration.

6 Generative method: GAN + DPS

The fourth, generative, method combines a GAN with a data-guided sampling loop inspired by Diffusion Posterior Sampling (DPS), adapted here to a GAN prior instead of a diffusion model (the difference is discussed in detail further on, Section 6.6). The method is organized into two clearly separate phases: (1) training an unconditional GAN generator, which learns only the distribution of plausible CT images, without ever seeing sinograms or measurements (Sections 6.1–6.5); (2) using that generator, frozen, as a prior inside a data-guided sampling loop (DPS) to reconstruct each test image from its sinogram (Sections 6.6–6.8). Both phases are complete; the quantitative results are discussed in Section 8 together with the other three methods, and the specific limitations of the method in Section 6.9.

6.1 Why two separate phases: the link with deep generative priors

The clear separation between training the generator and reconstructing a single image is the distinguishing feature of methods based on a *Deep Generative Prior* (DGP), and sharply distinguishes them from the end-to-end approach of Section 5. The UNet directly learns a map (degraded image) $\rightarrow$ (reconstructed image): at inference time it performs a single forward pass, without ever comparing its own output against the observed physical measurement. The GAN described in this section, on the contrary, learns no input $\rightarrow$ output map at all: the generator G_θ takes as input only noise $z \sim \mathcal{N}(0, I)$ (no degraded image, no sinogram) and produces plausible images of the CT domain. The prior knowledge that training injects into it is purely statistical ("what an image of this kind looks like"), not a mechanism for correcting a specific artifact.

It is only in phase (2) (Section 6.6) that this prior is made specific to a measurement: a latent code z is optimized such that the generated image $G_\theta(z)$, projected through the operator K (Section 3), is consistent with the specific test sinogram y_δ . The physical operator K never enters the generator's training (phase 1): it only enters the reconstruction loop (phase 2), exactly as in the general DGP framework. For this reason the weights saved in this phase (`generator_weights_best.pth`) are the only artifact that needs to be carried over to the subsequent notebooks: everything else (optimizers, discriminator, training history) was used only here and is discarded.

6.2 Architecture

The generator and discriminator follow the DCGAN scheme [?], via transposed/direct convolutions with stride 2, with no fully-connected layers:

- **Generator:** input $z \in \mathbb{R}^{100}$ (`latent_dim= 100`), treated as a $100 \times 1 \times 1$ tensor and progressively expanded through 7 layers of `ConvTranspose2d` (kernel 4, stride 2, padding 1, except the first layer, which expands from 1×1) up to $256 \times 256 \times 1$. The intermediate channels follow the progression $32 \cdot 16 \rightarrow 32 \cdot 8 \rightarrow 32 \cdot 4 \rightarrow 32 \cdot 2 \rightarrow 32 \rightarrow 16 \rightarrow 1$ (`ngf= 32`), with BatchNorm and ReLU after every layer except the last, where a sigmoid constrains the output to $[0, 1]$, consistent with the convention used throughout the project (Section 3);
- **Discriminator:** mirrored architecture, 6 layers of `Conv2d` with stride 2 (`ndf= 32`), BatchNorm and LeakyReLU (slope 0.2), no final sigmoid (`BCEWithLogitsLoss` is used, numerically more stable than the explicit sigmoid+BCE combination);
- **Initialization:** convolutional weights from $\mathcal{N}(0, 0.02^2)$, BatchNorm parameters from $\mathcal{N}(1, 0.02^2)$ (scale) and 0 (bias): standard DCGAN initialization, not PyTorch's default, chosen for adversarial-training stability.

6.3 Learning rate tuning

Unlike TV, UNet, and Weighted TV (Sections 4.3, 5, 7.3), here PSNR and SSIM are not applicable as a tuning criterion: the GAN has no (input, target) pairs: the generator produces plausible images starting from noise, it does not transform one specific image into another, so there is no single "correct" output to compare pixel by pixel. The standard criterion in the literature for comparing GAN configurations is **FID** (Fréchet Inception Distance): it measures how closely the statistical distribution of the generated images resembles that of the real images (computed in the feature space of a pre-trained Inception-v3 network; lower = better), accompanied by a visual inspection of the generated images to spot phenomena such as mode collapse that FID alone may not capture. FID is computed against the validation set (never the test set), the same separation principle already used for TV/UNet/Weighted TV.

Notebook `06_GAN_hyperparameter_tuning.ipynb`: 4 combinations of learning rate (generator, discriminator) trained briefly (15 epochs), Adam with `betas= (0.5,0.999)` (standard convention for GANs, instead of the default (0.9,0.999)), label smoothing on the real labels (0.9 instead of 1.0, to prevent the discriminator from becoming overly confident and ceasing to provide a useful gradient to the generator). Results in Table 2:

Table 2: GAN learning-rate tuning: FID after 15 epochs, per combination.

lr_G	lr_D	FID
$2 \cdot 10^{-4}$	$2 \cdot 10^{-4}$	385.09
$2 \cdot 10^{-4}$	$1 \cdot 10^{-4}$	396.58
$1 \cdot 10^{-4}$	$2 \cdot 10^{-4}$	403.73
$1 \cdot 10^{-4}$	$4 \cdot 10^{-4}$	519.20

The symmetric combination $\text{lr}_G = \text{lr}_D = 2 \cdot 10^{-4}$ (the DCGAN default) wins, albeit narrowly over the first two alternatives. The case $\text{lr}_D = 4 \cdot 10^{-4}$ (discriminator 4 times faster than the generator) is clearly the worst: in the loss curves of that run (available in the notebook), the discriminator loss shows an unstable spike around epoch 4 and the generator loss, after the initial drop, starts *rising* again until the end of training instead of stabilizing: the typical pattern of a discriminator that learns too fast relative to the generator: it quickly becomes too good at telling real from fake, the gradient it returns to the generator stops being informative, and the generator can no longer improve. This is why, in GANs, it is never a good idea to assign the discriminator a learning rate much higher than the generator's, even though intuitively "a stronger discriminator" might seem desirable.

6.4 Full training

Notebook `06_GAN_training_new.ipynb`, run on Google Colab (GPU L4): 150 epochs, batch size 32, the same hyperparameters as the tuning run (learning rate from the winning combination, label smoothing, DCGAN initialization). A complete checkpoint (weights of both networks, state of both optimizers, epoch, loss/FID history) is saved to Drive every 10 epochs, with automatic resume logic in case the Colab session disconnects (the same practical need already addressed for TV and Weighted TV, Section 4), more critical here since 150 epochs of GAN training require a long session.

Saving strategy: best FID, not the last epoch. Unlike the resume checkpoint (which is overwritten at every save), the generator weights in `generator_weights_best.pth` are overwritten **only when the measured FID improves** on the minimum observed so far. This choice is not redundant: Figure 10 shows that FID during training **does not** decrease monotonically. The absolute minimum is observed at epoch 60 (FID = 314.77), after which FID

worsens and oscillates for the rest of training (between 328 and 353), ending at epoch 150 at $\text{FID} = 328.32$, worse than the minimum reached 90 epochs earlier. The loss curves (same figure) explain why: after the rapid initial drop, the generator loss slowly starts rising again in the second half of training, while the discriminator's stays low and essentially flat: a sign that the discriminator maintains (or regains) a persistent advantage over the generator in the final part of training. This is typical behavior of adversarial training: unlike a single-objective optimization, the generator/discriminator min-max game has no guarantee of monotonic convergence, and can degrade after passing through a good point. Saving "always the last epoch", as would be done for TV/UNet/Weighted TV, would have silently discarded the best result of training here. The improvement over the short tuning run is nonetheless substantial: from $\text{FID} 385.09$ (15 epochs, Section 6.3) to $\text{FID} 314.77$ (60 epochs), a relative reduction of 18.3%, confirming that extending training beyond the short tuning phase was worth the extra computational cost.

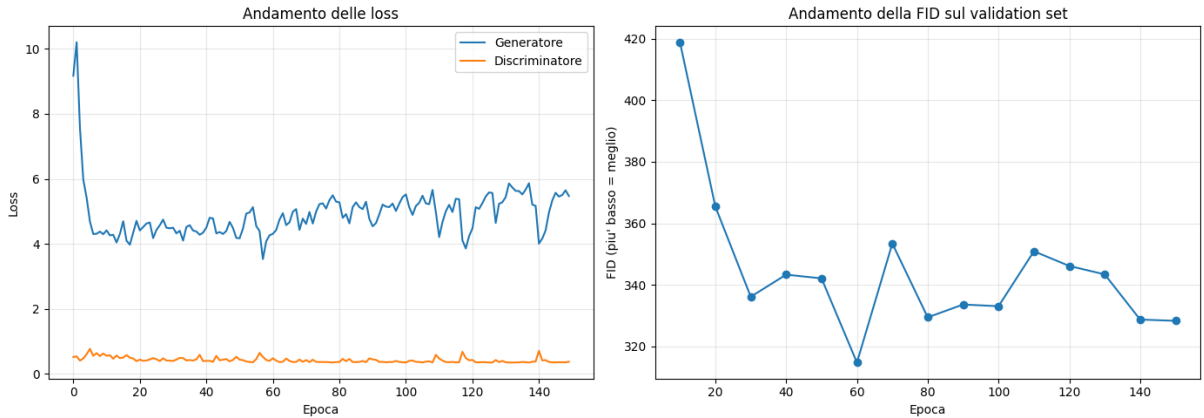


Figure 10: Evolution of loss (generator/discriminator) and of FID on the validation set during the 150 epochs of full training. The FID minimum (epoch 60) does not coincide with the last epoch.

6.5 Attempt with learning rate decay

The FID plateau/oscillation after epoch 60 (Section 6.4) motivated a second training attempt, in `06_GAN_training_decay.ipynb`: same architecture and same initial hyperparameters, but with lr_G and lr_D halved every 30 epochs starting from epoch 60 (so $2 \cdot 10^{-4} \rightarrow 1 \cdot 10^{-4} \rightarrow 5 \cdot 10^{-5} \rightarrow 2.5 \cdot 10^{-5} \rightarrow 1.25 \cdot 10^{-5}$), under the hypothesis that the plateau was due to steps that were too large near a local optimum, rather than to a structural imbalance between generator and discriminator.

The hypothesis does not hold up: Figure 11 shows that the best FID reached with decay is 331.96 (epoch 30, *before* the decay even kicks in), worse than the minimum 314.77 obtained without decay, and never surpassed again over the remaining 120 epochs despite the progressively smaller learning rate. The generator loss, instead of stabilizing, rises more sharply than in training without decay (up to 6.86 versus ~ 5.5 -5.9), while the discriminator's stays low and increasingly flat. The most plausible reading is that the problem was not the step size near an optimum, but a structural imbalance between the two networks (the discriminator systematically "wins"): reducing the learning rate slows down the generator's adaptation to this imbalance rather than correcting it, so the second half of training is in part computation time spent worse than in the original run. For this reason the DPS phase (Sections 6.6–6.8) uses the generator *without* decay (`gan_checkpoints/generator_weights_best.pth`, $\text{FID} 314.77$), not the one from this attempt.

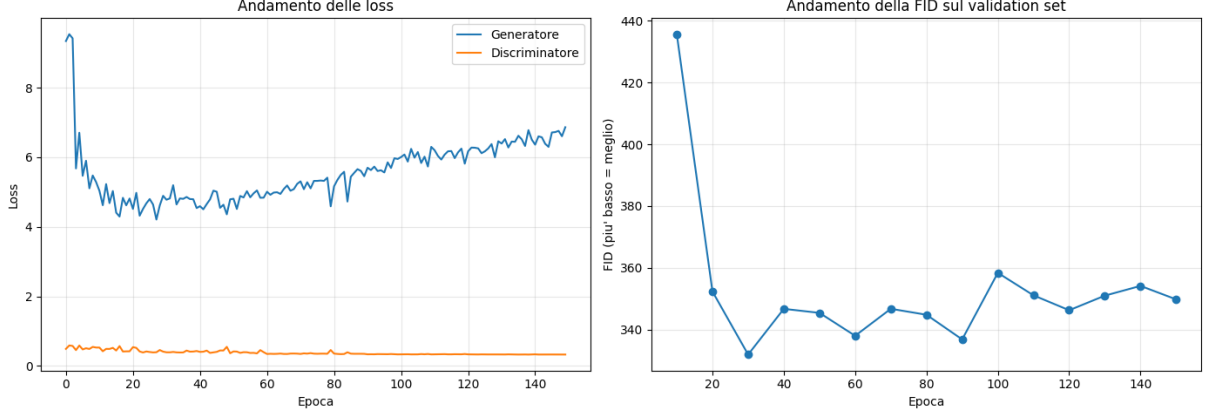


Figure 11: Evolution of loss and FID with learning rate decay (halved every 30 epochs starting from epoch 60). The FID minimum (epoch 30, 331.96) precedes the start of the decay and is never reached again: worse than the minimum 314.77 obtained without decay (Figure 10).

6.6 Diffusion Posterior Sampling: formulation

With the generator frozen (Section 6.4), the reconstruction of each test slice is obtained by optimizing the latent code z with respect to the objective

$$\mathcal{L}(z) = w_{\text{data}} \|K G_{\theta}(z) - y_{\delta}\|_2^2 + \|z\|_2^2, \quad (3)$$

where the first term is fidelity to the observed sinogram (exactly as in the TV discrepancy term, Section 4, but here the gradient with respect to z passes through the entire generator) and the second is a prior on z : since the generator was trained with $z \sim \mathcal{N}(0, I)$ (Section 6.1), $\|z\|_2^2$ is, up to constants, the negative log-likelihood of that standard Gaussian, and therefore penalizes latent codes far from the region where the generator was actually trained to produce plausible images.

z is updated with Adam (not fixed-step gradient descent: an initial fixed-step version gave unstable results, with the winning combination always at the edge of the grid tested during tuning, a symptom of a step poorly scaled across the components of z) for `N_ITERS` iterations, with the optional addition, after every Adam step, of a small amount of noise

$$z \leftarrow z + \sqrt{2\eta} \sigma_{\text{noise}} \xi, \quad \xi \sim \mathcal{N}(0, I), \quad (4)$$

with $\eta = \text{step_size}$ (Adam’s learning rate) and $\sigma_{\text{noise}} = \text{noise_scale}$. This term (which can be disabled with $\sigma_{\text{noise}} = 0$) is a Langevin-type dynamic, and brings the method closer to genuine posterior sampling instead of a purely deterministic optimization of (3) toward a single maximum a posteriori (MAP) point. It is important to be precise about what this actually implements: it is not DPS in the strict original sense (which requires a diffusion model with an explicit noise schedule and a network that estimates the score $\nabla_x \log p(x)$ at every noise level), but an adaptation of the same principle, latent-code optimization with data guidance plus a small stochastic term, to a standard GAN generator, which has neither an explicit schedule nor an explicit score. The final reconstruction is $x^* = G_{\theta}(z)$ after `N_ITERS` iterations.

6.7 DPS hyperparameter tuning

Notebook `06_GAN_DPS_hyperparameter_tuning_clean.ipynb`: grid search over `data_weight` (w_{data} in (3)), `step_size` and `noise_scale` (in (4)), the same protocol as the other tuning runs in the project (12 validation samples, combined score $0.5 \cdot \text{PSNR}_{\text{norm}} + 0.5 \cdot \text{SSIM}_{\text{norm}}$). Grid:

$$\text{data_weight} \in \{0.01, 0.1, 1.0\}, \quad \text{step_size} \in \{10^{-3}, 10^{-2}, 10^{-1}\}, \quad \text{noise_scale} \in \{0, 0.02\},$$

18 combinations per angular configuration, with `N_ITERS`= 300 already at tuning time: the same value later used in the final generation (Section 6.8), to avoid the tuning/generation misalignment already discussed for TV (Section 4.3, where instead the tuning run uses `maxiter` halved relative to generation).

The chosen parameters are reported in Table 3.

Table 3: DPS hyperparameters chosen per angular configuration (mean validation PSNR/SSIM of the winning combination).

Configuration (n. angles)	<code>data_weight</code>	<code>step_size</code>	<code>noise_scale</code>	PSNR / SSIM (val.)
90	0.01	10^{-2}	0.00	17.44 dB / 0.3274
45	0.01	10^{-2}	0.00	17.44 dB / 0.3277
30	0.10	10^{-3}	0.02	16.41 dB / 0.3065
15	0.10	10^{-3}	0.02	16.38 dB / 0.3056

A tuning finding more important than the chosen parameters. Within each configuration, the best combinations are nearly indistinguishable from one another even at the same `step_size`: for 90 angles, PSNR = 17.44 dB (to two decimal places) with `data_weight` = 0.01, with 0.1, and with 1.0 alike, a two-order-of-magnitude range on the weight of the data-fidelity term. In other words, **how strongly the reconstruction is forced to match the observed sinogram has almost no effect on the final result**, within the explored range. This is a diagnostic symptom, not merely a tuning detail: if increasing the weight of data fidelity by a factor of 100 changes (almost) nothing, reconstruction quality is not constrained by the fidelity/prior trade-off that Equation (3) is meant to balance, but by something further upstream: how well the generator manages to produce images that resemble real CT slices, regardless of which specific measurement it is asked to match. This point is taken up again in Section 6.9.

6.8 Generation

Notebook `06_GAN_DPS_generation.ipynb`, the same Colab/GPU/CuPy-patch scheme as the other generation runs (Section 4), `N_ITERS`= 300 (Section 6.7), parameters per configuration from Table 3, read automatically from the tuning file. As with Weighted TV, this notebook only produces the reconstructions: PSNR/SSIM against the ground truth are computed separately, with the same evaluation scheme as the other three methods (Section 8). Generation time: ~ 2.6 s per image (300 DPS iterations each), roughly one hour for the entire test set ($327 \text{ slices} \times 4 \text{ configurations} = 1308 \text{ reconstructions}$); no failures on any of the 1308 images. Same incremental sync logic to Drive, with automatic resume, already used for TV and Weighted TV.

6.9 Limitations of the method

The quantitative results (Section 8) place GAN+DPS clearly below the other three methods in every angular configuration, by a much wider margin than the one separating TV, UNet, and Weighted TV from one another. Three observations, independent but consistent with one another, point to where the bottleneck lies:

1. **High absolute FID.** 314.77 (Section 6.4) is a high value in absolute terms: well-trained generators on image-synthesis tasks typically reach FID values in the tens, not the hundreds. It is a direct signal of what, when discussing *Deep Generative Priors* in the abstract (Section 6.1), can be called *generator distortion*: the distribution learned by G_θ covers only part of, and imprecisely, the true distribution of CT images in the domain.

2. **Insensitivity to data_weight in DPS tuning** (Section 6.7). If the weight of the data-fidelity term can vary by two orders of magnitude while changing almost nothing in the result, the sampling loop is not really using the sinogram to steer the reconstruction toward a better solution: it is essentially returning (almost) the best the generator is able to produce, regardless of which specific measurement it is asked to match.
3. **Visual evidence** (Figure 14, Section 8): the GAN+DPS reconstructions are not sharp-but-different images relative to the ground truth (the behavior that the perception/distortion trade-off from the literature would lead one to expect), but instead show a grainy/noisy texture spread over the whole image, with only an approximate agreement in overall shape with the real anatomy. This is consistent with a generator that has not yet converged cleanly, not with an error in the z -optimization loop (which, by construction, checks the sinogram residual at every step).

The learning-rate-decay attempt (Section 6.5) rules out that the remedy was a gentler training schedule in the final phase: the limitation appears to be structural (a relatively simple DCGAN architecture, a training dataset of a few thousand images in a single anatomical domain, 150 epochs) rather than a fine-tuning problem. A realistic margin for improvement would more likely come from a more expressive generator or from more data/training epochs than from further tuning of `data_weight/step_size/noise_scale`: point 2 above shows that this lever has already been explored and is not where the quality of the result is decided. The overall computational cost, it must be said, is nonetheless the highest among the four methods: 150 epochs of GAN training, the additional DPS tuning, and ~ 1 hour of generation on the test set alone, for a result that is quantitatively inferior to the other three. It is nonetheless an informative result, not a failure of the pipeline: Section 8 takes it up again in the overall comparison.

7 Hybrid method: Weighted Total Variation

7.1 Formulation

The hybrid method follows Morotti et al. [3] (arXiv:2501.09845). The idea of the method is to replace the scalar regularization weight λ of global TV (Section 4) with a *per-pixel* weight w_i :

$$\text{TV}_w(x) = \sum_i w_i |Dx|_i, \quad (5)$$

where D is the discrete gradient operator. The weights are computed from an intermediate image $\tilde{x}$ (here, the output of the already-trained UNet, Section 5, applied to the FBP reconstruction of the same sinogram) via:

$$w_i = \left(\frac{\eta}{\sqrt{\eta^2 + |D\tilde{x}|_i^2}} \right)^{1-p}. \quad (6)$$

Flat regions of $\tilde{x}$ (gradient ≈ 0) receive a weight close to 1, hence full TV regularization; edges and details (high gradient) receive a weight close to 0, so regularization is relaxed and details are preserved. The final reconstruction x^* is obtained by solving the weighted variational problem (Chambolle-Pock) using the **real sinogram** y_δ as the data-fidelity term, exactly as in global TV: the UNet no longer directly produces the final image, but only guides the weight map. This distinguishes the method from a simple "TV(UNet output)" post-processing scheme: the final reconstruction remains physically consistent with the observed measurement, not only with the network's output.

7.2 Why the weights are fixed and not recomputed at every iteration

IPPy’s `ChambollePockTpVConstrained` solver already includes a reweighting mechanism (*iterative reweighting*, IR11 in the reference paper), which recomputes the weights at every iteration from the current solution. The reference paper, however, uses this scheme as the *baseline to beat*, with results clearly worse in its own tables than its own contribution: computing the weights **once** from the observed data (via the network) and keeping them fixed for the entire optimization. This is not merely a design choice but a structural assumption used by the authors to prove that the method is well posed. For this reason IPPy’s solver was not reused by simply passing `p<1`, but subclassed (`ChambollePockWeightedTpVConstrained`), reusing unchanged the entire primal-dual update logic of the original and changing only the point at which the weight map W is computed: outside the iteration loop instead of inside it.

7.3 Hyperparameter tuning

The method introduces two additional hyperparameters relative to global TV, η and p in Equation (6), besides λ . Tuning (notebook `05_WeightedTV_hyperparameter_tuning.ipynb`) follows the same protocol used for TV (Section 4.3): grid search over 12 validation samples per configuration, `maxiter`= 150 for the relative comparison between combinations, combined score $0.5 \cdot \text{PSNR}_{\text{norm}} + 0.5 \cdot \text{SSIM}_{\text{norm}}$. Explored grid:

$$\lambda \in \{0.01, 0.03, 0.05, 0.1, 0.2\}, \quad \eta \in \{10^{-3}, 2 \cdot 10^{-3}, 5 \cdot 10^{-3}, 10^{-2}\}, \quad p \in \{0.1, 0.3, 0.5\},$$

for a total of 60 combinations per angular configuration. The chosen parameters are reported in Table 4.

Table 4: Hyperparameters chosen for the Weighted TV method, per angular configuration.

Configuration (n. angles)	λ	η	p
90	0.20	$2 \cdot 10^{-3}$	0.1
45	0.20	$5 \cdot 10^{-3}$	0.1
30	0.03	$1 \cdot 10^{-3}$	0.5
15	0.01	$5 \cdot 10^{-3}$	0.3

A pattern observable in Table 4 is that the configurations with more angles (90, 45) select $p = 0.1$ (exponent $1 - p = 0.9$ in Equation (6), hence a strong contrast between the weights of flat regions and of edges), while the configurations with fewer angles (30, 15) select a higher p (0.5 and 0.3, a softer contrast). A plausible reading is that the weight map w_i depends on the gradient of the intermediate image $\tilde{x}$ produced by the UNet (Equation (6)), and the UNet itself is less reliable precisely in the configurations with fewer angles (Section 8, UNet-vs-ground-truth comparison): with a noisier $\tilde{x}$, a soft contrast between weights (high p) is a more cautious choice than a sharp one, because it does not amplify confidence in edges that the intermediate image may have estimated poorly.

7.4 Generation

The Weighted TV reconstructions on the test set are generated with notebook `05_WeightedTV_generation_tune` the same Colab/GPU/CuPy-patch scheme as TV (Section 4, subsection "Implementation details"), with `maxiter`= 300, the same value and the same quality/time trade-off discussed for TV in Section 4.3.

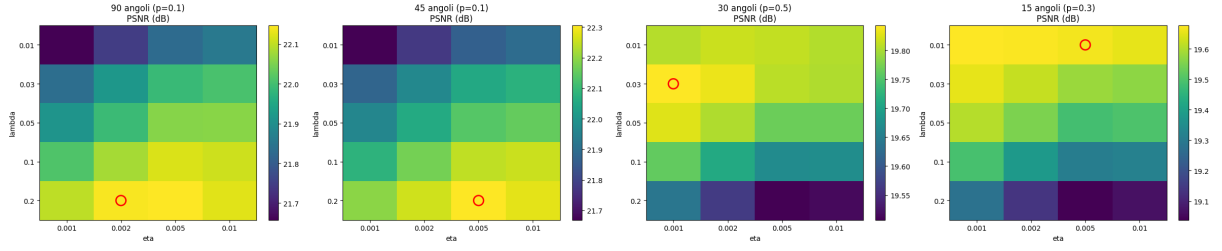


Figure 12: Heatmap of mean PSNR as λ and η vary (at the best p), for each angular configuration; the circled point indicates the chosen combination.

7.5 Limitations of the method

Unlike TV, whose only additional cost is tuning λ , Weighted TV inherits two non-trivial dependencies. The first is on the UNet: the weight map w_i (Equation (6)) comes from the network trained in Section 5, so any systematic bias of that network (for example a localized inaccuracy on a certain type of edge or structure) propagates directly into the regularization, instead of remaining confined to a separate output as in the UNet-vs-GT comparison of Section 8. The second is computational cost: unlike the UNet, which reduces inference to a single forward pass, Weighted TV remains an iterative solver with the same `maxiter` as global TV (Section 4.3) plus the additional cost of an upstream UNet inference, so it offers no speed advantage over plain TV: the gain in Section 8 is purely in quality, not in computation time.

8 Results

8.1 Experimental setup

All methods are evaluated on the same test set (327 slices of patient C081, Section 2) and on the same degraded sinograms (Section 3), for each of the 4 angular configurations, via PSNR and SSIM with respect to the ground truth: notebook 03_TV_results.ipynb for TV, 04_UNet_results.ipynb for UNet, 05_WeightedTV_results_tuned.ipynb for Weighted TV. The notebook 07_final_comparison.ipynb aggregates the summaries produced by these three and generates the table and figures of this section.

8.2 Quantitative comparison

Table 5 reports mean PSNR and SSIM ($\pm$ standard deviation over the 327 slices) for each method and angular configuration.

Four observations emerge from the table:

1. **Weighted TV is the best method in all 4 configurations**, both in PSNR and in SSIM, with a consistent but modest gain over global TV (between +0.11 and +0.23 dB of PSNR, between +0.008 and +0.012 of SSIM). The gain is systematic across all angular configurations, not an isolated effect of a single case, which supports the method’s central hypothesis: spatially adaptive regularization, guided by a learned prior, helps more than uniform regularization alone or the learned prior alone.
2. **The UNet does not exceed TV on the final metric, and this is the expected behavior, not an anomaly.** The UNet-vs-TV comparison (not reported in Table 5, available in the evaluation notebooks) shows a very high fidelity to its own training target: PSNR between 28.7 and 30.7 dB and SSIM between 0.88 and 0.90 in all configurations, versus the ~ 21 – 23 dB / ~ 0.6 – 0.7 of the final metric (vs. ground truth); see also the visual comparison in Figure 13, where the two reconstructions are nearly indistinguishable. As

Table 5: Quantitative comparison between the methods with respect to the ground truth, per angular configuration. 327 slices, patient C081.

Method	Metric	90 angles	45 angles	30 angles	15 angles
TV	PSNR (dB)	23.83 ± 2.12	23.79 ± 2.14	21.75 ± 2.35	21.51 ± 2.37
	SSIM	0.7257	0.7221	0.6426	0.6231
UNet	PSNR (dB)	23.35 ± 1.64	23.32 ± 1.48	21.15 ± 1.98	20.91 ± 2.12
	SSIM	0.7189	0.7153	0.6250	0.6139
GAN+DPS	PSNR (dB)	16.05 ± 1.43	15.97 ± 1.40	15.48 ± 1.29	15.30 ± 1.35
	SSIM	0.2914	0.2893	0.2845	0.2799
Weighted TV	PSNR (dB)	24.06 ± 2.10	23.94 ± 2.07	21.91 ± 2.36	21.62 ± 2.27
	SSIM	0.7376	0.7309	0.6506	0.6299

anticipated in Section 5, since the network is trained to imitate TV and not the ground truth, its quality with respect to the ground truth is bounded above by that of TV itself: an imperfect imitation (however good, as here) can only subtract quality relative to that bound, not add to it. The data confirm exactly this: the UNet is nearly indistinguishable from its own target but ends up slightly below TV on the ground truth ($-0.2/-0.6$ dB depending on the configuration). It follows that the "end-to-end approach that imitates a variational method" is not, in itself, a way to *improve* that method: it is only a way to *approximate it at a much lower inference cost* (a single forward pass versus 300 Chambolle-Pock iterations). The hybrid method of Section 7 overcomes this structural limitation in a precise way: it does not use the network to produce the final image (so it is not bound by the quality of the training target), but only to estimate a weight map that then guides a solver anchored to the real sinogram: this is why it manages to exceed TV, while the plain UNet does not.

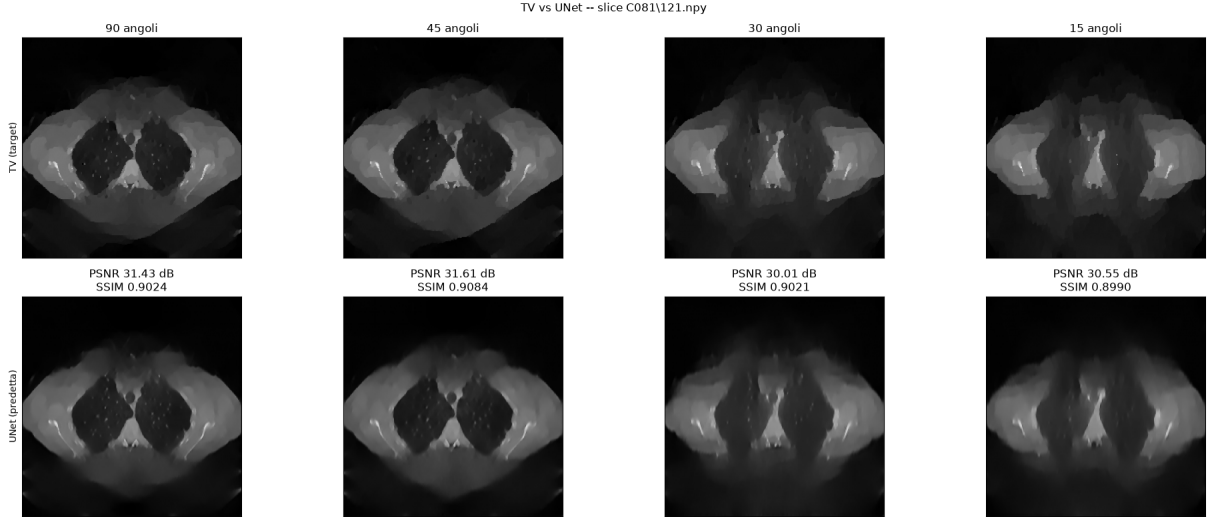


Figure 13: TV reconstruction (training target) and UNet output compared, same slice, all angular configurations: the two reconstructions are visually almost indistinguishable (SSIM > 0.89 in every configuration).

- GAN+DPS is clearly the weakest method**, with a gap much wider than the one separating the other three methods from one another: from -6.2 to -8.0 dB of PSNR relative to TV depending on the configuration, SSIM less than half. The reasons are

discussed in detail in Section 6.9: high absolute FID, insensitivity of the result to the weight of data fidelity in DPS tuning, and a visibly noisy texture in the reconstructions (Figure 14) all point to the same bottleneck, the capacity of the generator itself rather than a problem in the sampling loop.

4. **The UNet has the lowest standard deviation among the pixel-optimized methods** (1.48–2.12 dB of PSNR, versus 2.07–2.37 for TV and Weighted TV), despite having the lowest mean. This is a typical effect of training with an MSE-dominant loss: the network tends to produce "smoother" and more uniform output across slices, sacrificing some peak quality for greater consistency. TV and Weighted TV, solving an optimization problem independently for each individual image, have greater variability but reach higher peak values on the most favorable slices.

8.3 Qualitative comparison

Figure 14 shows, for the same slice of patient C081 and all 4 angular configurations, the visual comparison between ground truth, TV, UNet, GAN+DPS, and Weighted TV (notebook `07_final_comparison.ipynb`, file `results/final_comparison/final_visual_comparison_all_methods.png`). This is where the limitation of the generative method discussed in Section 6.9 is more clearly visible than in any number: the GAN+DPS column shows a grainy texture spread over the entire image, clearly distinguishable by eye from the other three, with only the overall anatomical shape recognizable.

8.4 Overall comparison between methods

To compare all methods at a glance we produced a plot of mean PSNR and SSIM as the number of angles varies, one curve per method (Figure 15, file `results/final_comparison/comparison_psnr_ssim_all_methods.png`). The top row shows all 4 methods on a full scale: this is where one can see at a glance how far apart GAN+DPS is from the other three. Precisely because it occupies such a much lower band of values, on a full scale the differences between TV, UNet, and Weighted TV, the ones that matter for the main comparison in Section 7, would end up compressed into a narrow band; the bottom row repeats the same plot restricted to these three methods, with the vertical axis auto-scaled to their range alone, to make them readable.

As a complement, Figure 16 shows the full distribution of PSNR and SSIM over the 327 slices (not just mean and std), a boxplot per method grouped by angular configuration: more informative than the mean curve alone, because it also makes the variability between slices visible, not just the central value. Here too GAN+DPS occupies a band completely separate from the other three, with no overlap between the boxes.

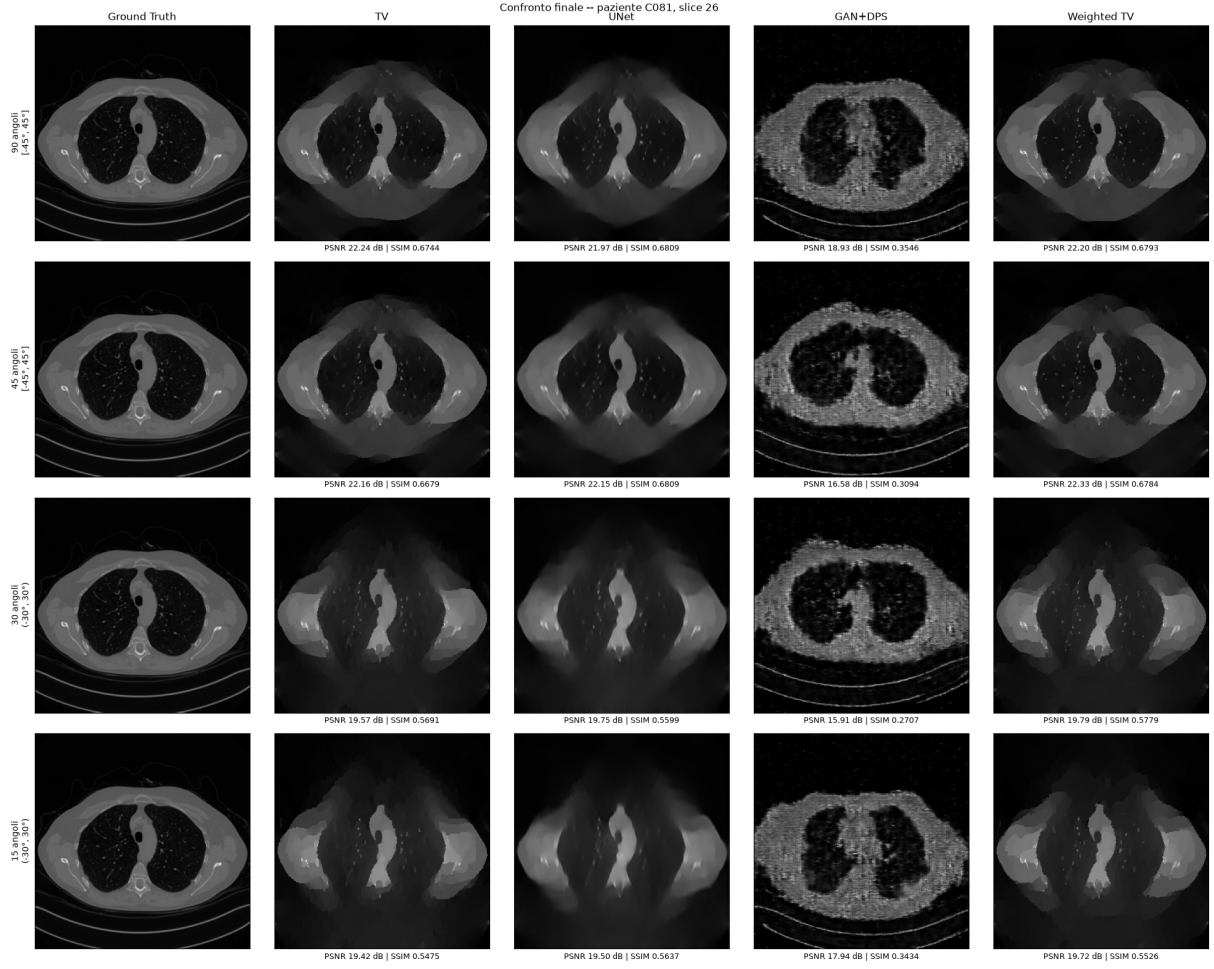


Figure 14: Visual comparison: ground truth, TV, UNet, GAN+DPS, and Weighted TV, same slice of patient C081, all angular configurations (rows) with PSNR/SSIM for each reconstruction.

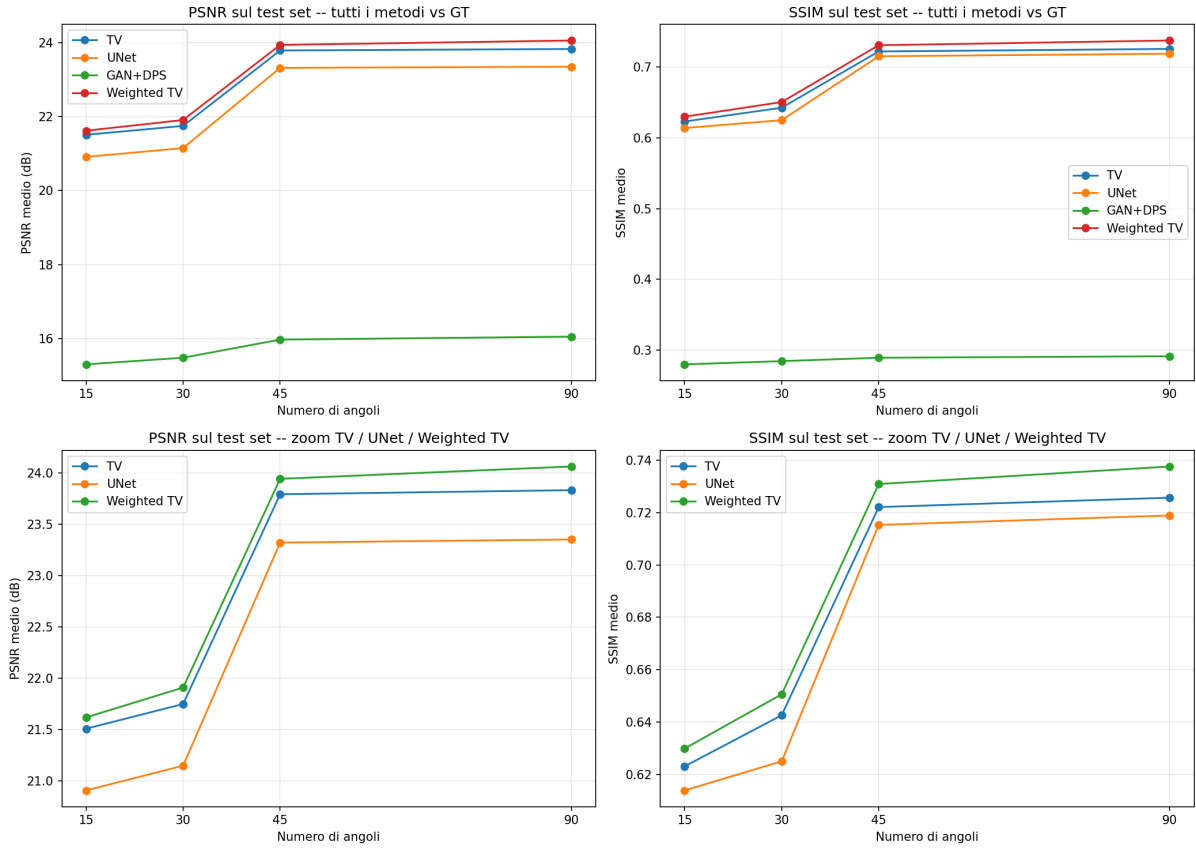


Figure 15: Top: mean PSNR and SSIM on the test set as the number of angles varies, all 4 methods. Bottom: same comparison, zoomed on TV/UNet/Weighted TV (GAN+DPS excluded, to make the differences between the three competitive methods readable).

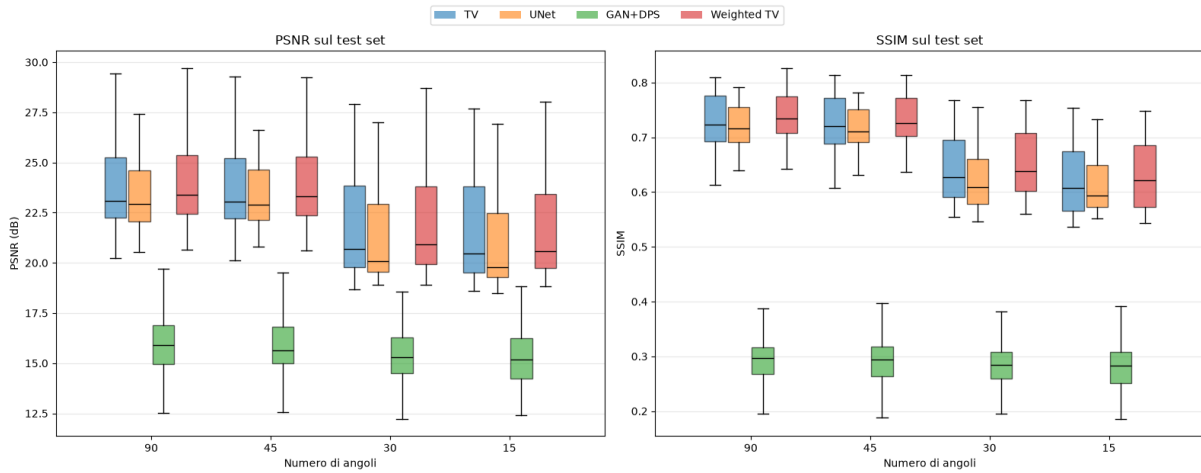


Figure 16: Distribution of PSNR and SSIM over the 327 slices of the test set, per method and angular configuration.

9 Conclusions

The comparison between the four implemented methods (TV, UNet, GAN+DPS, Weighted TV) on the same test set indicates a consistent and stable ranking across all angular configurations: **Weighted TV $\geq$ TV $>$ UNet $\gg$ GAN+DPS**, both in PSNR and in SSIM. The most significant result from a methodological point of view is not so much the absolute gain of Weighted TV (modest, ~ 0.1 – 0.2 dB), but its *systematic nature* across all 4 configurations, which supports the basic idea of the hybrid method: a learned prior can improve a classical regularization if used to guide its spatial structure (weight map), rather than to replace it entirely (as in the pure end-to-end approach, UNet, or the pure generative approach, GAN+DPS). The UNet, in this comparison, plays a role different from the one initially hypothesized: it does not exceed TV, but is a fast and effective approximation of it (fidelity to the target > 0.88 SSIM), useful where inference cost matters more than marginal quality. GAN+DPS, despite being the only one of the four methods whose generator is trained on the ground truth rather than on an intermediate target, fails to exploit this theoretical advantage: the quality of the learned prior, as measured by FID, is the dominant constraint (Section 6.9).

As physically expected, the reconstruction quality of TV, UNet, and Weighted TV degrades monotonically as the number of available angles decreases (the *missing-wedge* problem becomes increasingly ill posed), with a sharper drop between 45 and 30 angles than between the other pairs of adjacent configurations. GAN+DPS also degrades monotonically, but over a much narrower range (less than 1 dB of PSNR between 90 and 15 angles, against more than 2 dB for the other three): consistent with Section 6.9, its quality is dominated by the generator’s capacity rather than by the specific difficulty of the angular configuration.

Limitations of the study. Some limitations must be explicitly acknowledged, some structural and others solvable with additional work:

- **Single-patient test set:** all quantitative metrics come from a single CT volume (327 slices of patient C081, Section 2); the observed ranking between methods has not been verified on other patients;
- **Number of TV (and Weighted TV) iterations:** fixed at 300 due to computation-time constraints, while tuning suggests that quality has not yet stabilized at 400 iterations (Section 4.3); the reported results may therefore slightly underestimate the potential of both variational methods;
- **Generative method complete but clearly weaker:** despite two generator training attempts (with and without learning rate decay, Sections 6.4–6.5) and a dedicated tuning of the DPS hyperparameters (Section 6.7), GAN+DPS remains the weakest method in every configuration. The limitation appears to depend on the generator’s capacity (training dataset of limited size, relatively simple DCGAN architecture, 150 epochs) rather than on suboptimal tuning of the sampling loop (Section 6.9);

References

- [1] Davide Evangelista. Ippy: Inverse problems in python. <https://github.com/devangelista2/IPPy>. GitHub Repository.
- [2] Cynthia H. McCollough et al. Low dose ct grand challenge. Mayo Clinic. Dataset per esperimenti di tomografia.
- [3] Elena Morotti et al. Adaptive weighted total variation boosted by learning techniques in few-view tomographic imaging. *arXiv preprint arXiv:2501.09845*, 2025.